

Best Practice: Git Development Workflow

Disusun oleh: Vincentius Bryan Kwandou — bahan diskusi tim

Tanggal: 17 Juli 2026

Dokumen ini adalah usulan workflow Git untuk project tim. Poin-poinnya terbuka untuk didiskusikan dan disesuaikan bersama.

1. Struktur Branch

```
main ██████████●██████████●██████████ (selalu deployable)
\ /
develop ██████████●███●███●██████████ (integrasi harian)
\ /
feature/login-jwt █●███● (satu fitur = satu branch)
```

- `main` — kode production. Hanya menerima merge dari `develop` (atau `hotfix`). Selalu dalam kondisi bisa di-deploy.
- `develop` — branch integrasi. Semua feature branch bermuara ke sini.
- `feature/<nama-fitur>` — satu branch per fitur/tugas, dibuat dari `develop`. Contoh: `feature/login-jwt`, `feature/dashboard-crud`.
- `fix/<nama-bug>` untuk perbaikan bug, `hotfix/<nama>` dari `main` untuk perbaikan darurat production.

Aturan: tidak ada commit langsung ke main dan develop — semuanya lewat Merge Request.

2. Konvensi Commit

Pakai format **Conventional Commits** supaya history mudah dibaca dan bisa di-generate changelog:

```
<type>: <deskripsi singkat, kalimat perintah>
```

```
feat: add jwt login endpoint
fix: use pg driver so app connects to local postgres
docs: add docker setup guide
chore: bump next.js to 16.2
refactor: extract auth middleware
test: add register validation tests
```

Prinsip:

- Commit kecil dan atomik — satu commit satu perubahan logis.
- Pesan menjelaskan **kenapa**, bukan hanya apa (detail bisa di body commit).
- Jangan commit file hasil build (`.next/`, `node_modules/`) — pakai `.gitignore`.
- **Jangan pernah commit secret** (`.env`, token, connection string). Sediakan `.env.example` sebagai template.

3. Alur Kerja Harian

1. `git pull origin develop` — mulai hari dengan kode terbaru.
2. `git checkout -b feature/nama-fitur` — buat branch untuk tugas hari ini.
3. Kerjakan → commit kecil-kecil secara berkala.
4. `git push -u origin feature/nama-fitur` — push minimal sekali sehari (backup + visibilitas tim).

5. Buka **Merge Request** ke `develop` saat fitur siap (atau draft MR lebih awal untuk feedback).
6. Minimal **1 reviewer** approve sebelum merge. Reviewer mengecek: logika, keamanan, dan apakah ada file yang tidak seharusnya ikut.
7. Setelah merge, hapus feature branch.

4. Merge Request (MR) yang Baik

- Judul jelas: `feat: login page dengan JWT`.
- Deskripsi berisi: apa yang berubah, cara test, screenshot bila menyentuh UI.
- Ukuran kecil ($< \sim 400$ baris perubahan) — MR besar sulit direview, lebih baik dipecah.
- Selesaikan konflik di branch sendiri (`git pull origin develop` lalu `rebase/merge`) sebelum minta review.
- CI harus hijau (`build + lint + test`) sebelum merge — bisa dipasang lewat `.gitlab-ci.yml`.

5. Yang Sebaiknya Dihindari

Anti-pattern	Kenapa
Commit langsung ke <code>main</code>	Melewati review, berisiko merusak production
<code>git push --force</code> ke branch bersama	Menimpa pekerjaan orang lain
Commit "wip", "update", "fix lagi" berulang	History tidak informatif
Satu commit raksasa di akhir hari	Sulit direview dan di-revert
Menyimpan credential di repo	Bocor permanen di history git, harus rotate
Branch hidup berminggu-minggu tanpa sync	Konflik besar saat merge

6. Perintah yang Sering Dipakai

```
git status # selalu cek sebelum add/commit
git add -p # stage per bagian, hindari file nyasar
git commit -m "feat: ..." # commit dengan pesan jelas
git pull --rebase origin develop # sync tanpa merge commit berantakan
git log --oneline --graph # lihat riwayat bercabang
git stash / git stash pop # simpan sementara kerjaan belum selesai
```

7. Proteksi di GitLab (usulan setting project)

- Protected branch: `main` dan `develop` (no force push, merge via MR only).
- MR approval minimal 1 orang.
- Pipeline CI wajib lulus sebelum merge.
- Personal Access Token per orang untuk push HTTPS (bukan share satu token).

Catatan: workflow ini berdasarkan pengalaman minggu pertama (multi-remote, protected branch, token auth di GitLab) digabung praktik umum industri. Silakan dikoreksi/ditambah saat diskusi tim.